

## Multiclass Classification

**Multiclass classification** is a type of classification problem where the output variable,  $y$ , can be one of **three or more** possible discrete categories (or classes).

This is an extension of binary classification, where  $y$  could only be 0 or 1.

---

### Examples of Multiclass Classification 📁

- **Handwritten Digit Recognition:** Classifying an image as one of the 10 digits (0, 1, 2, 3, 4, 5, 6, 7, 8, or 9).
  - **Medical Diagnosis:** Identifying which of several possible diseases a patient has (e.g., "Disease A," "Disease B," "Disease C," or "Healthy").
  - **Manufacturing Defects:** Inspecting a product and classifying its defect type (e.g., "Scratch," "Chip," "Discoloration," or "No Defect").
- 

### Contrast with Binary Classification

- **Binary Classification:**
    - Has two output classes (e.g., 0 or 1).
    - The model (logistic regression) estimates a single probability:  $P(y = 1)$ .
  - **Multiclass Classification:**
    - Has  $N$  (where  $N > 2$ ) output classes (e.g., 1, 2, 3, 4).
    - The model must estimate the probability for **each class**:  $P(y = 1)$ ,  $P(y = 2)$ ,  $P(y = 3)$ , and  $P(y = 4)$ .
    - The model learns decision boundaries that divide the input space into multiple regions, one for each class.
- 

### Softmax Regression: The Algorithm for Multiclass

To handle multiclass problems, we need a new algorithm. **Softmax regression** is a generalization of logistic regression that can handle more than two classes. This will be the key component used to build neural networks for multiclass classification.

## Softmax Regression

**Softmax regression** is the algorithm used for **multiclass classification** problems (where  $y$  can be one of  $N$  possible classes, with  $N > 2$ ). It is a direct generalization of logistic regression.

---

### From Logistic to Softmax Regression

To understand softmax, it helps to first re-frame logistic regression:

- **Logistic Regression (Binary:  $N = 2$ )**
    - Calculates one linear output:  $z = \vec{w} \cdot \vec{x} + b$ .
    - Calculates one probability:  $a_1 = \text{sigmoid}(z) = P(y = 1)$ .
    - The second probability is implicitly defined:  $a_2 = 1 - a_1 = P(y = 0)$ .
    - The probabilities  $a_1$  and  $a_2$  must sum to 1.
  - **Softmax Regression (Multiclass:  $N \geq 2$ )**
    - The model must compute a separate probability for *each* of the  $N$  classes:  
 $a_1, a_2, \dots, a_N$ .
    - These  $N$  probabilities must sum to 1.
- 

### The Softmax Regression Model

The model's computation is a two-step process:

#### Step 1: Compute Linear Outputs ( $z_j$ )

- Instead of one set of parameters, the model must learn **one set of parameters ( $\vec{w}_j, b_j$ ) for each class  $j$** .
- It computes a separate linear output (or "score") for each class:
  - $z_1 = \vec{w}_1 \cdot \vec{x} + b_1$
  - $z_2 = \vec{w}_2 \cdot \vec{x} + b_2$
  - ...
  - $z_N = \vec{w}_N \cdot \vec{x} + b_N$

#### Step 2: Compute Activations ( $a_j$ ) using the Softmax Function

- We cannot use the sigmoid function, as the resulting probabilities would not sum to 1.
- We use the **softmax function**, which takes the vector of  $z$ -scores and converts it into a vector of probabilities.
- The formula for the activation (probability) of a single class  $j$  is:

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}}$$

- **What this does:**

1. By using  $e^{z_j}$ , it ensures all outputs are positive.
2. By dividing by the sum of all  $e^{z_k}$ , it ensures the final probabilities  $a_1, a_2, \dots, a_N$  **all sum to 1**.

- **Interpretation:**  $a_j$  is the model's estimated probability that the true label  $y$  is equal to class  $j$ , given the input  $\vec{x}$ .
- 

## Loss Function for Softmax Regression

This model uses a loss function called **cross-entropy loss**.

- **Logistic Loss (Recap):**  $L = -y \log(a_1) - (1 - y) \log(a_2)$
- **Softmax (Cross-Entropy) Loss:** This generalizes the logistic loss. The loss for a single training example is calculated based *only* on the probability the model assigned to the **single correct class**.
  - If the true label  $y = 1$ , then Loss =  $-\log(a_1)$
  - If the true label  $y = 2$ , then Loss =  $-\log(a_2)$
  - ...
  - If the true label  $y = j$ , then Loss =  $-\log(a_j)$
- **Intuition:** To make the loss small (close to 0), the model must learn to make the probability  $a_j$  for the *correct* class  $j$  as close to 1 as possible.
- **Cost Function:** The overall cost  $J$  is the average of this loss function  $L$  over all  $m$  training examples.

**Note:** Softmax regression with  $N = 2$  is mathematically equivalent to logistic regression.

# Neural Networks for Multiclass Classification

To adapt a neural network for a multiclass classification problem (e.g., classifying digits 0–9), we modify the **output layer**. Instead of using a single logistic (sigmoid) unit, we use the **softmax regression** model as the final layer.

---

## The Softmax Output Layer

### Architecture

The architecture of the hidden layers remains the same (e.g., using ReLU activations). The change is in the output layer:

- **Binary Classification:** The output layer has **1 unit** (neuron) with a **sigmoid** activation.
- **Multiclass Classification:** The output layer has **N units** (neurons), one for each class. For digit recognition (0–9), this means the output layer will have **10 units**. This is called a **softmax layer**.

### Forward Propagation

1. **Hidden Layers:** The activations for the hidden layers ( $\vec{a}^{[1]}$ ,  $\vec{a}^{[2]}$ , etc.) are computed exactly as before (e.g., using ReLU).
2. **Output Layer ( $\vec{a}^{[3]}$ ):**

- **Step 2a (Compute Z):** The final layer first computes a linear score,  $z_j$ , for each of the 10 output units. The input is the activation vector from the previous layer,  $\vec{a}^{[2]}$ .

$$z_j^{[3]} = \vec{w}_j^{[3]} \cdot \vec{a}^{[2]} + b_j^{[3]} \quad (\text{for } j = 1 \text{ to } 10)$$

- **Step 2b (Compute A):** The **softmax function** is then applied *across all 10 z values* to convert these scores into probabilities.

$$a_j^{[3]} = P(y = j | \vec{x}) = \frac{e^{z_j^{[3]}}}{\sum_{k=1}^{10} e^{z_k^{[3]}}}$$

The result,  $\vec{a}^{[3]}$ , is a vector of 10 probabilities (one for each class) that will sum to 1.

---

## A Key Property of Softmax

Unlike the sigmoid, ReLU, and linear activation functions, the softmax activation is **not element-wise**.

- For sigmoid/ReLU, the output of a single neuron ( $a_j$ ) depends *only* on its corresponding linear input ( $z_j$ ).
- For **softmax**, the output of a single neuron ( $a_j$ ) depends on **all** the linear inputs (

$z_1, z_2, \dots, z_{10}$ ) because of the sum in the denominator.

---

## TensorFlow Implementation

The 3-step process in TensorFlow is updated as follows:

### 1. Define the Model

The output layer is changed to a Dense layer with N units and the 'softmax' activation.

```
1 model = Sequential([
2     Dense(units=25, activation='relu'),
3     Dense(units=15, activation='relu'),
4     # Output layer has 10 units and softmax activation
5     Dense(units=10, activation='softmax')
6 ])
```

### 2. Compile the Model

We must use a new loss function designed for multiclass classification:

**SparseCategoricalCrossentropy**.

```
1 model.compile(loss='sparse_categorical_crossentropy')
```

- **CategoricalCrossentropy**: This is the TensorFlow name for the softmax loss function ( $L = -\log(a_j)$ ) that we saw previously.
- **Sparse**: This refers to the format of your true labels,  $y$ . It means your  $y$  labels are **single numbers** (e.g., 0, 1, 7, 9). This is in contrast to a "one-hot" format.

### 3. Train the Model

This step remains exactly the same as before.

```
1 model.fit(X, Y, epochs=...)
```

---

## Important Warning

**Do not use the code exactly as written above.**

While this code *will* work, it is **not** the recommended or most numerically stable version. A "better" version, which is more accurate, will be shown in the next video.

## A Better Implementation: Numerical Stability

There is a more advanced and numerically stable way to implement neural networks for classification. This recommended method helps prevent a common issue called **numerical round-off error**.

### The Problem: Numerical Round-off Error

Computers have a limited amount of precision to store numbers (known as floating-point numbers). Certain calculations can lead to small, accumulating errors.

- **Simple Analogy:**
  - **Direct (Accurate) Way:**  $x = 2 / 10000$
  - **Indirect (Less Accurate) Way:**  $x = (1 + 1/10000) - (1 - 1/10000)$
- Mathematically, these are identical. But in a computer, the "indirect" method can introduce tiny round-off errors.
- **Applying this to Neural Networks:**
  1. The "indirect" method is computing the activation  $a = \text{sigmoid}(z)$  *first* and *then* feeding  $a$  into the loss function.
  2. The "direct" method involves giving the raw  $z$  value to the loss function and letting it perform a combined, more stable calculation.

### The `from_logits` Solution

The recommended implementation involves setting `from_logits=True` in the TensorFlow loss function.

- **What are "logits"? Logits** are the raw, linear outputs of the final layer,  $z$ , **before** the activation function (like sigmoid or softmax) is applied.
  - **How it works:** By setting `from_logits=True`, you tell TensorFlow: "I am giving you the  $z$  values, not the  $a$  values. Please combine the activation function and the loss calculation into one mathematically stable step."
  - **Why is this better for Softmax?** The softmax function uses  $e^z$ . If  $z$  is a large number,  $e^z$  can become astronomically large (an "overflow"). If  $z$  is a large negative number,  $e^z$  can become vanishingly small (an "underflow"). The `from_logits=True` method uses mathematical tricks to compute the loss *without* computing these huge or tiny intermediate numbers, making it much more accurate.
-

## Recommended TensorFlow Implementation

Here is a comparison of the standard (less stable) and recommended (more stable) methods.

| Problem                   | Standard (Less Stable)                                                                                 | Recommended (More Stable)                                                                                           |
|---------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Binary Classification     | <pre>Dense(units=1, activation='sigmoid') model.compile(loss='binary_crossentropy')</pre>              | <pre>Dense(units=1, activation='linear') model.compile(loss=BinaryCrossentropy(from_logits=True))</pre>             |
| Multiclass Classification | <pre>Dense(units=10, activation='softmax') model.compile(loss='sparse_categorical_crossentropy')</pre> | <pre>Dense(units=10, activation='linear') model.compile(loss=SparseCategoricalCrossentropy(from_logits=True))</pre> |

### Important Consequence: Making Predictions

There is one critical side effect of using the recommended `from_logits` method:

- Your network's output is **no longer a probability**. The output layer now has a `linear` activation, so it outputs the raw **logits** ( $z$ ).
- If you need to get a probability (e.g., to show to a user), you must **manually apply the activation function** to the model's output *after* making a prediction.
  - For logistic, you would pass the output  $z$  through a sigmoid function.
  - For softmax, you would pass the output vector  $\vec{z}$  through a softmax function.

# Multi-Label Classification

A **multi-label classification** problem is one where a single input,  $X$ , can be associated with **multiple output labels** at the same time.

This is different from multi-class classification, where each input has only *one* label, even if that label can be one of many possible values (e.g.,  $y = 7$ ).

## Example: Autonomous Driving 🚗

A common example is scene detection for a self-driving car.

- **Input ( $X$ ):** An image from the car's camera.
- **Task:** Identify all objects present in the image.
- **Output Labels ( $y$ ):** The output is a **vector** of  $N$  binary labels, one for each possible object.
  - Is there a car? (Yes/No)
  - Is there a bus? (Yes/No)
  - Is there a pedestrian? (Yes/No)

A single image could have an output vector of  $y = [1, 0, 1]$ , meaning "Car" and "Pedestrian" are present, but "Bus" is not.

## Multi-Label vs. Multi-Class

This table summarizes the key difference:

| Feature      | Multi-Class Classification                        | Multi-Label Classification                                  |
|--------------|---------------------------------------------------|-------------------------------------------------------------|
| Example      | Handwritten Digit (0-9)                           | Scene Detection (Car, Bus, Pedestrian)                      |
| Output $y$   | A single number (e.g., $y = 7$ )                  | A vector of numbers (e.g., $y = [1, 0, 1]$ )                |
| Question     | "Which <i>one</i> of these classes is this?"      | "Which <i>of these classes</i> (zero or more) are present?" |
| Output Layer | <b>Softmax</b> (forces probabilities to sum to 1) | <b>Sigmoid</b> (applied to each neuron independently)       |



## Neural Network Implementation

You can build a single neural network to solve a multi-label problem.

- **Hidden Layers:** These are standard (e.g., using `relu` activations).
- **Output Layer:** This is the key difference.
  - The output layer must have **N neurons**, where N is the number of labels you are trying to predict (e.g., 3 neurons for car/bus/pedestrian).
  - The activation function for *each* of these neurons must be **sigmoid**.

This effectively treats the problem as N separate binary classification problems that the network learns to solve simultaneously.